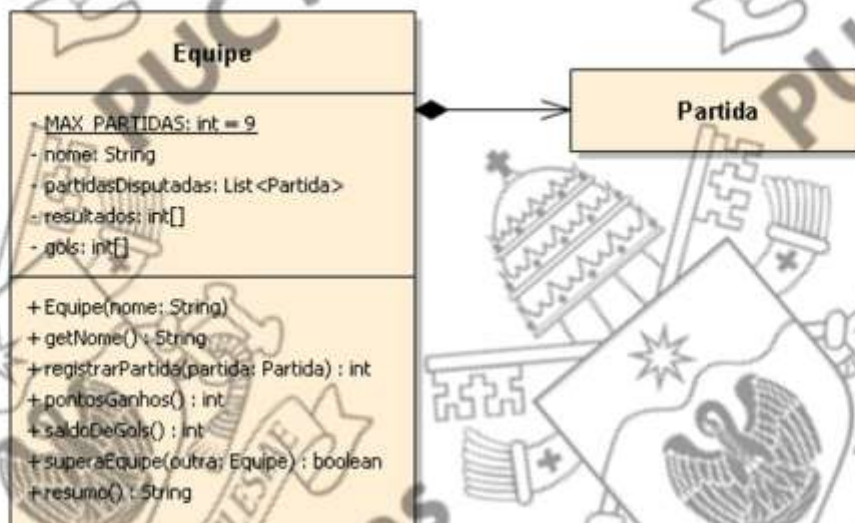


1) método construtor da classe Partida.

```
public Partida(String mandante, String visitante,
               String placar){
    if(mandante.length == 0)
        mandante = "Mandante";
    if(visitante.length == 0)
        visitante = "Visitante";
    equipes[0] = mandante;
    equipes[1] = visitante;
    String[] gols = placar.split("x");
    this.placar[0] = this.placar[1] = 0;
    if(gols.length == 2){
        this.placar[0] = Integer.parseInt(gols[0]);
        this.placar[1] = Integer.parseInt(gols[1]);
    }
}
```



2)



SEM VETOR/LISTA PARA ARMAZENAR ESTATÍSTICAS DA EQUIPE

3a) o registro de uma partida para uma equipe.

```
public int registrarPartida(Partida partida){
    boolean verificado = nome.equals(partida.mandante());
    if(!verificado())
        verificado = nome.equals(partida.visitante());

    if(partidasDisputadas.size() < MAX_PARTIDAS && verificado){
        partidasDisputadas.add(partida);
    }

    return partidasDisputadas.size();
}
```

3b) a totalização de pontos obtidos por uma equipe no campeonato.

```
public int pontosGanhos(){
    int total = 0;
    for(Partida partida : partidasDisputadas){
        int pontos = 0;
        if(partida.vencedor.equals(nome))
            pontos = 3;
        else if(partida.vencedor.equals("empate"))
            pontos = 1;
        total += pontos;
    }
    return total;
}
```

COM VETOR PARA ARMAZENAR AS ESTATÍSTICAS DA EQUIPE

RESULTADOS = {VITÓRIAS, EMPATES, DERROTAS}

GOLS = {MARCADOS, SOFRIDOS}

3a) o registro de uma partida para uma equipe.

```
public int registrarPartida(Partida partida){
    int pos = nome.equals(partida.mandante()) ? 0 : -1;
    if(pos == -1)
        pos = nome.equals(partida.visitante()) ? 1 : -1;
    if(partidasDisputadas.size() < MAX_PARTIDAS && pos != -1){
        String adversario = (pos == 0) ? partida.visitante() : partida.mandante();
        partidasDisputadas.add(partida);
        gols[0] = partida.golsDaEquipe(nome);
        gols[1] = partida.golsDaEquipe(adversario);
        int resultado = partida.ganhador().equals(nome) ? 0 : 2;
        if(partida.ganhador().equals("empate"))
            resultado = 1;
        resultados[resultado]++;
    }
    return partidasDisputadas.size();
}
```

3b) a totalização de pontos obtidos por uma equipe no campeonato.

```
public int pontosGanhos(){
    return resultados[0] * 3 + resultados[1];
}
```

4) teste unitário JUnit para um dos dois métodos da questão (3)

```
@Test
public void calculaPontosGanhosCorretamente(){
    Equipe equipe = new Equipe("Xulams");
    Partida empate = new Partida("Xulams", "ProgMod", "2x2");
    Partida vitoria = new Partida("Xulams", "LPM", "4x1");
    Partida derrota = new Partida("Xulams", "FPAA", "0x1");
    equipe.registrarPartida(empate);
    equipe.registrarPartida(vitoria);
    equipe.registrarPartida(derrota);

    int pontos = equipe.pontosGanhos();

    assertEquals(4, pontos);
}
```